

Winter of Code 2025/26

This file contains the report of kaggle competition, Binary Buzz , which was a classification problem.

Understanding the datasets

The competition provided 2 files : `train.csv` and `test.csv`

train.csv

This file had a shape of (2000, 3074) :

2000 - 2000 rows or datasamples were provides

3074 - 3074 columns which induced the training features, id column and the target column.

Key Highlights about the file (All of the features are collapsible and have code and a small summary) :

- **Loading the data**
Loading the `train.csv` dataset using pandas as storing it as a dataframe.

```
```python import pandas as pd df = pd.read_csv('train.csv') ``` The dataset is stored as a dataframe using a vriable df
```

</details>

- **Duplicate columns**  
Checking for any duplicate columns , if any ( would drop them)

```
df.duplicated().sum() #> 0
```

No duplicate columns were found

</details>

- **Splitting the dataset**  
Splitting the dataset into features and target variable / independent and dependedt variable

```
X = df.drop(columns=['id','target'], axis = 1) y = df['target'] # axis = 1 , operates horizontally (on columns) X
```

X - features variable and y - target variable.

</details>

- **Train Test Split**  
Splitting the data into training and testing data chunks to avoid data leakage in the model which evaluating the models or potential overfitting of the model.

```
from sklearn.model_selection import train_test_split X_train , X_val , y_train , y_val = train_test_split(X,y,tes
```

Used sci-kit learn library's model\_selection's feature train\_test\_split , to split the data.

</details>

- **Correlation Matrix**  
Checking for highly correlated matrix and dropping them.

```
corr = X_train.corr().abs() # Returns a square matrix with corr of each column with other columns upper = corr.wh
```

Dropped almost 303 columns , which had correlation higher than 0.95.

</details>

- **Outlier Detection**

Outliers are datapoints which are very different from rest of the data / they don't fit the normal patterns. Using the InterQuartile Range (IQR)

```
Q1 = X.quantile(0.25) # 25 percentile Q3 = X.quantile(0.75) # 75 percentile IQR = Q3 - Q1 outliers_count = ((X < (
```

Did not remove the outliers because they dont affect tree models , which are distance independent. (eg: XGBoost, LightGBM, RandomForest, etc...)

</details>

- **Standard Scaler**

Using Standard Scaler to rescale the data such that all features lies in similar ranges and have mean 0 and standard deviation as 1.

```
from sklearn.preprocessing import StandardScaler scaler = StandardScaler() X_train = scaler.fit_transform(X_train
```

</details>

## Experiments and Trials

Most of my experiments were done to get rid of the high dimensionality of the data (Curse of Dimensionality).

All of the below points are collapsible and can be opened by clicking them.

- **ANOVA**

ANOVA stands for Analysis of Variance , it used to check how well a feature seprates a class , so we can select the k best features (based on best split) out of 3072

```
from sklearn.feature_selection import SelectKBest, f_classif selector = SelectKBest(f_classif, k=1000) # k could
```

Tried to play around with different values of k and different models.

</details>

- **PCAs - Principal Component Analysis**

PCA combines many related features into fewer new features that capture most of the data's variation. This was implemented to

```
from sklearn.decomposition import PCA pca = PCA(n_components=0.85) X_train = pca.fit_transform(X_scaled) X_val =
```

Tried PCA with a variance of 0.85 , did not get much better results with them.

</details>

- **RFE-Recursive Feature Elimination**

RFE is a feature selection method that repeatedly trains a model, removes the least important features, and keeps the best ones.

```
from sklearn.feature_selection import RFE from sklearn.linear_model import LogisticRegression model = LogisticReg
```

Gave few good results , but tooke a lot of time to complete (~35-40 mins) , so couldn't experiment much.

</details>

The dataset had heavy class imbalance

class(0) : class(1) - 9:1 , to counter this did a few experiments :

- **Sampling - SMOTE**

SMOTE (Synthetic Minority Over-sampling Technique) ,it is an oversampling techinque which creates synthetic data of minority class.

```
from imblearn.over_sampling import SMOTE smote = SMOTE(random_state=42) X_train_resample, y_train_resample = smot
```

Did not improve the results much (~ - 0.2- 0.3) , felt like the models were overfitting sometimes.

</details>

- **Class weights**

Most of the models had the feature of adding class weights , which enables the learning of models to pay more attention to minority classes

```
#Logistic Regression model = LogisticRegression(class_weight={0: 1, 1: 5}) #RandomForest model = RandomForestClas
```

Imporved the balanced accuracy a lot , kept implemeting over the course of competition.

</details>

I even tried to tune my threshold value , which actually boosed my balanced accuracy score (learning from this I even implemeted a .probabilty in my ml\_model from scratch).

- **Threshold tuning (Best Balanced accuracy)**

SMOTE (Synthetic Minority Over-sampling Technique) ,it is an oversampling techinque which creates synthetic data of minority class.

```
from sklearn.metrics import balanced_accuracy_score import numpy as np thresholds = np.arange(0.0, 1.01, 0.01) be
```

Helped me increase the balanced accuracy from the one which had a default threshold as 0.5.

</details>

I tried hyperparameter tuning , which helped me figure out the best hyperparameters of my model:

- **GridSearchCV**

This tries out each and every hyperparamter on the model and returns the best set of hyperparameters , it does all the permuation and combinations of hyperparameters (mostly for small datasets).

```
from sklearn.ensemble import RandomForestClassifier from sklearn.model_selection import GridSearchCV rf = RandomF
```

Took a lot of time to implement , could not even complete the hyperparameter tuning using this , as I had a very large dataset to handle.

</details>

- **RandomSearchCV**

RandomSearchCV randomly picks few combinations and retuns the best hyperparameters from the combinations picked.

```
from sklearn.ensemble import RandomForestClassifier from sklearn.model_selection import RandomizedSearchCV rf = R
```

Was way faster than GridSearchCV and imporved the results significantly.

</details>

- **Optuna**

Used Optuna library for hyperparameter tuning which use bayesian statistics , which is basicly an intelligent way (rather than random) to pick combinations . Felt this a bit too new and complex , did try to implement but was running into errors, but it is a faster and more efficient method.

</details>

Tried experimenting with the a new way of validating the models performance

- **K-Fold Cross-Validation**

K-Fold Cross-Validation is a method to check how well your model really performs by testing it on different parts of the data, not just one split. It basically breaks the data into K sets , where K-1 sets go to training and the remaining one is used for evaluating the model.

```
from sklearn.model_selection import StratifiedKFold, cross_val_score cv = StratifiedKFold(n_splits=5, shuffle=True
```

</details>

---

## 'ID' Feature

So I found a strong correlation between the 'id' column and the target column , which boosted my balanced accuract , f1 , recall , precision. But eventually it was not a correct method or way (so dropped it).

---

### Final Submissions and python notebooks

XGB\_HP\_ANOVA.ipynb

Model Used : XGBoost

```
model = XGBClassifier(n_estimators=500, max_depth=10, learning_rate=0.03, subsample=0.75, colsample_bytree=0.7, colsam
```

Threshold tuning : Best Threshold - 0.06

```
from sklearn.metrics import balanced_accuracy_score import numpy as np thresholds = np.arange(0.0, 1.01, 0.01) best_thr
```

Metric	Score
Balanced Accuracy	0.6805555555555556
F1 score	0.32857142857142857
Precision	0.23
Recall	0.575
Public Score	1.0
Private Score	0.50087

...

LGBM\_HP\_ANOVA.ipynb

Model used : LightGBM

```
model = LGBMClassifier(random_state=42)
```

ANOVA : Used (k = 1000)

```
from sklearn.feature_selection import SelectKBest, f_classif selector = SelectKBest(f_classif, k=1000) X_train = select
```

HyperParameter tuning : RandomSearchCV

```
from sklearn.model_selection import RandomizedSearchCV from sklearn.metrics import make_scorer param_dist = { 'n_estima
```

Metric	Score
Balanced Accuracy	0.6513888888888889
F1 score	0.3053435114503817
Precision	0.21978021978021978
Recall	0.5
Public Score	1.0
Private Score	0.5002

Added Even the best model on the private dataset

RF\_HP\_ANOVA.ipynb

Model Used : Random Forest

```
model = RandomForestClassifier(n_estimators=1000, random_state=42)
```

ANOVA : K = 300

```
from sklearn.feature_selection import SelectKBest, f_classif selector = SelectKBest(f_classif, k=300) X_train = selecto
```

Threshold tuning : Best Threshold : 0.1

```
from sklearn.metrics import balanced_accuracy_score import numpy as np thresholds = np.arange(0.0, 1.01, 0.01) best_thr
```

HyperParameter tuning : RandomSearchCV

```
from sklearn.model_selection import RandomizedSearchCV from sklearn.metrics import make_scorer param_dist = { 'n_estima
```

Metric	Score
Balanced Accuracy	0.6944444444444444
F1 score	0.2714
Precision	0.15833333333333333
Recall	0.95
Public Score	1.0
Private Score	0.7099

Metrics of other models and values I tried

Model	k_selectKBest	Corr Threshold	SMOTE	Balanced Accuracy	Precision	Recall	F1 Score	Leaderboard Score	Hype
Random Forest	300	0.95	No	0.7166666667	0.2012987013	0.775	0.3195876289	0.99	HP
XGBoost	1000	0.95	Yes	0.6847222222	0.1904761905	0.70	0.2994652406	0.87	HP
Random Forest	1000	0.90	Yes	0.6763888889	0.1693989071	0.775	0.2780269058		HP
Random Forest	1000	0.95	No	0.6750000000	0.1562500000	0.875	0.2651515152		HP
Random Forest	1000	0.95	Yes	0.6611111111	0.1439393939	0.95	0.2500000000	0.93	HP
XGBoost	300	0.95	Yes	0.6500000000	0.1956521739	0.45	0.2727272727		HP
Random Forest	500	0.95	No	0.7083333333	0.1923076923	0.75	0.3061224490		HP
Random Forest	300	0.90	No	0.7097222222	0.1967213115	0.75	0.3114754098		HP
LightGBM	500	0.95	No	0.7222222222	0.2153846154	0.75	0.3349514563	1.02	HP
LightGBM	1000	0.90	Yes	0.7013888889	0.1984126984	0.775	0.3162393162		HP
LightGBM	300	0.95	Yes	0.6958333333	0.2058823529	0.70	0.3181818182		HP

Model	k_selectKBest	Corr Threshold	SMOTE	Balanced Accuracy	Precision	Recall	F1 Score	Leaderboard Score	Hype
CatBoost	1000	0.90	Yes	0.7125000000	0.2191780822	0.775	0.3417721519		HP